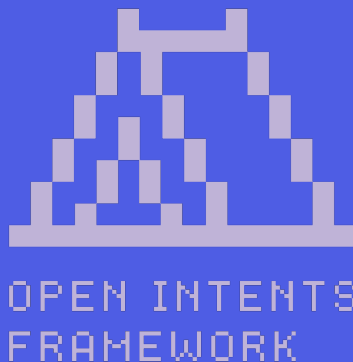


OIF Contracts Diff Audit



May 20, 2026

This document has been prepared solely for the named client and it may not be relied upon by any third parties. The content in this document is provided for informational purposes only and does not constitute professional advice of any kind. Security assessments are limited by factors such as scope, time, and techniques used and therefore this document does not constitute a certification, guarantee, or warranty regarding the proper functioning of any system or the absence of security vulnerabilities.

Copyright © 2026 Zeppelin Group Ltd.

Table of Contents

Table of Contents	2
Summary	3
Scope	4
System Overview	5
Hyperlane Oracle Chain ID Mapping	5
Permit2 Witness Type Update	5
InputSettlerCompact Order Status	5
Output Oracle Cleanness Check	6
FillerDataLib Length Validation	6
Security Model and Trust Assumptions	6
Medium Severity	8
M-01 Solver Identifier Encoding Mismatch Allows Bypass of Order Purchases	8
Low Severity	9
L-01 Reentrant Native ETH Transfer Can Drain Solver Excess in fill	9
L-02 purchaseOrder Does Not Validate Order Expiry	10
L-03 Missing Zero-Address Validation for orderPurchase.destination	10
L-04 Oversized Fill Descriptions Cannot Be Relayed Through MessageEncodingLib	10
Notes & Additional Information	12
N-01 Unused and Drifted Code in Permit2WitnessType	12
Conclusion	13
Appendix	14
Issue Classification	14

Summary

Type	Library	Total Issues	6 (6 resolved)
Timeline	From 2026-04-30 To 2026-05-01	Critical Severity Issues	0 (0 resolved)
Languages	Solidity	High Severity Issues	0 (0 resolved)
		Medium Severity Issues	1 (1 resolved)
		Low Severity Issues	4 (4 resolved)
		Notes & Additional Information	1 (1 resolved)
		Client Reported Issues	0 (0 resolved)

Scope

OpenZeppelin performed a diff-audit of the [openintentsframework/oif-contracts](#) repository, between base commit [222989b](#) and target commit [5323b1b](#). [This](#) diff highlights all the changes made between the two commits.

In scope were the following files:

```
src
├── input
│   ├── InputSettlerPurchase.sol
│   ├── compact
│   │   └── InputSettlerCompact.sol
│   └── escrow
│       ├── InputSettlerEscrow.sol
│       └── Permit2WitnessType.sol
├── integrations
│   └── oracles
│       └── hyperlane
│           ├── HyperlaneOracle.sol
│           └── HyperlaneOracleMapped.sol
└── output
    ├── OutputSettlerBase.sol
    └── simple
        └── FillerDataLib.sol
```

Update: The fixes for the findings highlighted in this report have all been merged at commit [b260675](#).

System Overview

The Open Intents Framework (OIF) is a modular, intent-based cross-chain swap protocol. Users sign intents that describe a desired asset delivery, and solvers compete to fulfill those intents on the requested output chain in exchange for the user's locked inputs on the source chain. The system separates input collection from output delivery, with a cross-chain oracle layer responsible for attesting that outputs were filled before inputs are released.

The changes under review address findings from the prior audit, harden cross-chain identifier handling, and introduce a new Hyperlane oracle variant with explicit chain ID mapping. The most notable changes are summarized below.

Hyperlane Oracle Chain ID Mapping

A new `HyperlaneOracleMapped` contract composes `ChainMap` with `HyperlaneOracle`, translating Hyperlane domain identifiers to canonical EVM chain IDs before storing attestations. The base `HyperlaneOracle` exposes a virtual `_getChainId` hook that returns the input identifier as is by default, and the mapped variant overrides it to look up the configured chain ID. Attestations are now stored under the resolved chain ID, aligning the storage key with what users sign in `MandateOutput.chainId`.

Permit2 Witness Type Update

The Permit2 witness used by `InputSettlerEscrow` was rebuilt to bind the order's `user` field as the first witness component. The witness typestring was simultaneously brought into line with EIP-712 sub-type ordering and with the canonical `bytes callbackData` field name in `MandateOutput`. The change closes a path where a Permit2 signature could be reused with a different `order.user`, and resolves a previously reported EIP-712 conformance issue with the typestring concatenation.

InputSettlerCompact Order Status

The Compact settler now tracks an `OrderStatus` per `orderId` with `None` and `Claimed` states. The status is checked and set inside `_resolveLock` before the external

`COMPACT.batchClaim` call, providing a checks-effects-interactions guard against replayed claims. The `purchaseOrder` flow additionally rejects already-claimed orders. A new `_transferInput` virtual lets the Compact settler extract the underlying ERC-20 from an ERC-6909 token ID during the discount transfer, and reverts on the native token, which the discount transfer path does not handle.

Output Oracle Cleanness Check

`OutputSettlerBase._fill` now rejects fills where `output.oracle` carries dirty upper bits. Without this gate, a solver could fill the output (paying the recipient) but find the cross-chain proof unsatisfiable, because the attester reconstructs the payload hash with `bytes32(uint256(uint160(msg.sender)))`, which is always clean, while the user's signed payload hash uses the dirty bytes. The new check fails closed at fill time, preventing the self-grief.

FillerDataLib Length Validation

`FillerDataLib.solver` now reverts with `FillerDataTooShort` when `fillerData` is shorter than 32 bytes, replacing an unchecked `calldataLoad`. The check is defense in depth, since equivalent unclaimable fills remain reachable with a full 32-byte non-clean solver value.

Security Model and Trust Assumptions

The OIF protocol is permissionless at the user and solver layers, and relies on a small number of trust boundaries for cross-chain liveness and correctness. The diff under review preserves the existing trust model and tightens the boundaries around cross-chain identifier handling.

In addition to trust assumptions from previous audits, we note that the owner of `HyperlaneOracleMapped` is honest and configures the chain ID map correctly before any user traffic flows. The `HyperlaneOracleMapped` contract inherits `Ownable` and exposes a single privileged operation, `setChainMap`, which records a `(protocolChainIdentifier, chainId)` pair. Each pair can only be configured once, enforced by zero-value and already-

set guards. Ownership transfer is single-step, consistent with the protocol's existing convention.

Medium Severity

M-01 Solver Identifier Encoding Mismatch Allows Bypass of Order Purchases

The `purchasedOrders` mapping is keyed by the full `bytes32` solver identifier, while `_purchaseOrder` verifies the purchase signature against `orderSolvedByIdentifier.fromIdentifier()`, which discards the upper 96 bits. The `OrderPurchase` EIP-712 type does not bind any solver identifier, so a single signature accepts every `bytes32` encoding that shares its lower 160 bits with the signing address. On the output side, the solver identifier flows verbatim from `fillerData` and the cross-chain attestation commits to exactly one `bytes32` encoding.

A malicious solver can exploit this asymmetry as follows:

1. The solver fills the order with `fillerData.solver = E1`, where `E1` carries non-zero upper bytes. The output settler attests to `E1`.
2. The solver signs an `OrderPurchase` for the order. The signature does not commit to `E1`.
3. A buyer calls `purchaseOrder` with `orderSolvedByIdentifier = E2`, where `E2` shares the lower 160 bits with `E1` but differs in the upper 96. `_purchaseOrder` verifies the signature against `E2.fromIdentifier()`, accepts it, writes the buyer into `purchasedOrders[E2][orderId]`, and transfers the discount tokens to `orderPurchase.destination`.
4. The solver calls `finalise` with `solveParams[0].solver = E1`. `_purchaseGetOrderOwner` reads `purchasedOrders[E1][orderId]`, finds an empty slot, and returns `solver = E1`. `_orderOwnerIsCaller` checks `E1.fromIdentifier() == msg.sender`, the check passes, and the solver finalizes the order to a destination of its choosing.

The buyer's discount payment is lost and the inputs are claimed by the solver. A buyer can defend against this by independently deriving the exact `bytes32` solver from the fill attestation and using it as `orderSolvedByIdentifier`, but the protocol provides no on-chain enforcement of that match. Because each distinct `bytes32` encoding hits a distinct storage slot, the same solver signature can be replayed against multiple buyers using different encodings, and the `AlreadyPurchased` guard only fires on a same-slot collision.

Consider canonicalizing the solver identifier across every boundary that records or verifies a purchase. Preferred options are requiring `LibAddress.validatedCleanAddress` on `orderSolvedByIdentifier` in `_purchaseOrder` and on `solveParams[i].solver` in `_purchaseGetOrderOwner`, or keying `purchasedOrders` by the address derived from the identifier rather than by the full `bytes32`. Adding a canonical `bytes32 solver` field to the `OrderPurchase` EIP-712 type would bind the signature to one encoding, but on its own still relies on the buyer checking that the signed solver matches the attested one, and is therefore weaker than canonicalizing at the storage and verification boundaries.

Update: Resolved in [pull request #176](#). The OpenZeppelin development team stated:

We solved the issue by using the clean solver identifier (keeping the solver as `bytes32` but cleaning potential dirty bits) `purchasedOrders` mapping, both on `_purchaseOrder` and `_purchaseGetOrderOwner`.

Low Severity

L-01 Reentrant Native ETH Transfer Can Drain Solver Excess in `fill`

The `OutputSettlerBase.fill` function is idempotent: when called for an already-filled output, `_fill` returns early instead of reverting, and `fill` then calls `refundNativeExcess`, which sends the current `address(this).balance` to `msg.sender`. For native ETH outputs, `_fill` delivers the output amount to the specified recipient using `Address.sendValue`, which forwards all remaining gas. If the recipient is a malicious contract, it can reenter `fill` with the same arguments. During this inner call, `_fill` returns early, and `refundNativeExcess` is called again, draining the solver's unspent ETH excess to the attacker. As a result, when the outer refund executes, the contract balance is already zero, so the solver receives nothing.

Consider tracking the refund amount as the initial `msg.value` minus the ETH transferred during execution, rather than reading `address(this).balance` after reentrancy. This removes reliance on the mutable contract balance, thereby mitigating this class of reentrancy risk.

Update: Resolved in [pull request #177](#).

L-02 `purchaseOrder` Does Not Validate Order Expiry

Neither `InputSettlerCompact.purchaseOrder` nor `InputSettlerEscrow.purchaseOrder` checks that `order.expires` has not passed before accepting a purchase. A purchaser can therefore pay for an order whose claim deadline has already elapsed. On the Compact settler, the subsequent `batchClaim` reverts at the TheCompact level, and the purchaser's payment is already with the solver. On the Escrow settler, the user can call `refund` before the purchaser finalizes, again leaving the purchaser's payment with the solver. The existing `expiryTimestamp` parameter guards only against the buyer's own transaction mining late, not against purchasing an already-expired order.

Consider validating that the order's expiry has not passed when purchasing an order.

Update: Resolved in [pull request #178](#).

L-03 Missing Zero-Address Validation for `orderPurchase.destination`

`_purchaseOrder` transfers input tokens to `orderPurchase.destination` without checking that it is non-zero, while `purchaser` is validated against the zero value. For tokens that do not revert on transfers to the zero address, the purchaser's payment may be irretrievably lost, as tokens sent to the zero address are effectively burned.

Consider adding a zero-address check for `orderPurchase.destination` alongside the existing `purchaser` validation.

Update: Resolved in [pull request #179](#).

L-04 Oversized Fill Descriptions Cannot Be Relayed Through `MessageEncodingLib`

`MandateOutputEncodingLib.encodeFillDescription` bounds `callbackData.length` and `context.length` individually against `type(uint16).max` but does not cap the total encoded length. The encoded fill description is 168 fixed bytes plus the two body fields, so any combination where `callbackData.length + context.length > 65367` produces a payload longer than `type(uint16).max`. The relay step in `HyperlaneOracle._submit` routes payloads through

`MessageEncodingLib.encodeMessage`, which prefixes each payload with a `uint16` length and rejects any payload longer than `type(uint16).max` with `TooLargePayload`.

A malicious user can open an order whose output sets `recipient` to themselves, leaves `context` empty, and inflates `callbackData` past 65367 bytes. A solver who fills the output transfers the output amount to the user and runs the oversized callback, then calls `submit` to relay the fill description and reverts inside `encodeMessage`. With no cross-chain attestation, the order expires, the user refunds the inputs, and the solver absorbs both the output amount and the fill gas with no recourse on the input chain.

Same-chain attestation via `setAttestation` is unaffected because it does not pass through `MessageEncodingLib`. The issue is specific to oracles that relay raw fill descriptions through that library: `HyperlaneOracle` and `WormholeOracle` exhibit it directly on the submit side. `BroadcasterOracle.submit` hashes payloads and broadcasts a digest rather than calling `encodeMessage`, but its destination `verifyMessage` derives payload hashes through `MessageEncodingLib.getHashesOfEncodedPayloads`, so an oversized payload cannot be represented there either.

Consider enforcing the total encoded length in `encodeFillDescription` so that `callbackData.length + context.length` cannot exceed `type(uint16).max - 168`. As an in-scope alternative, consider rejecting oversized payloads at the entry to `HyperlaneOracle.submit` so the failure is reported up front, or refusing to fill the output in `OutputSettlerBase._fill` when its encoded fill description would exceed the relay budget.

Update: Resolved in [pull request #180](#). The OpenZeppelin development team stated:

After consideration, we decided to not do the check for `MAX_FILL_DESCRIPTION_BODY` onchain, but rather only documenting it in the natspec and warn solvers that they should do this check offchain before filling an intent. This was chosen because the check was an edge case that added additional gas and wasn't solving the issue, as orders may have multiple outputs and simply doing the check offchain is enough for the solver not be harm.

Notes & Additional Information

N-01 Unused and Drifted Code in Permit2WitnessType

The `Permit2Witness struct` declares three fields, while the typestring constants and `Permit2WitnessHash` hash four fields including the `address user`. The struct is not used for hashing, so the drift has no on-chain effect, but it misleads readers. Separately, `PERMIT2_WITNESS_TYPE_STUB` is declared but never read throughout the codebase.

Consider updating `Permit2Witness` to include `address user` as the first field to match the typestring, and removing `PERMIT2_WITNESS_TYPE_STUB` if it is not required by any off-chain consumers.

Update: Resolved in [pull request #181](#). The OpenZeppelin development team stated:

The PR adds `address user` to the struct for consistency.

`PERMIT2_WITNESS_TYPE_STUB` is kept for offchain customers who might need it.

Conclusion

The audited diff hardens cross-chain identifier handling in the OIF protocol, introduces a new Hyperlane oracle variant with explicit chain id mapping, and resolves previously reported issues including an EIP-712 conformance gap in the Permit2 witness typestring and a missing `order.user` binding in that witness. It also tightens fill-time validation in the output settler against dirty bits in `output.oracle`, adds an `OrderStatus` guard to `InputSettlerCompact`, and corrects a regression in the Compact purchase path that prevented purchasing ERC-6909-encoded orders.

No Critical or High severity issues were identified. The review surfaced one Medium severity issue, a solver identifier encoding mismatch that lets a malicious solver bypass purchase ownership tracking. Four Low severity issues complete the body of the report: a reentrancy in the native-ETH refund path of `OutputSettlerBase`, two `purchaseOrder` hygiene gaps (a missing order-expiry check and a missing zero-address check on `orderPurchase.destination`), and a missing payload size check that allows a malicious user to produce fills that cannot be relayed through `MessageEncodingLib`. One informational note on documentation drift in `Permit2WitnessType` accompanies the findings.

The codebase remains well-structured and the diff continues the modular separation between input settlement, output settlement, and the oracle layer established in the prior audit. The OpenZeppelin Development team for Open Intents Framework was responsive throughout the engagement and provided clear context around the design intent behind cross-chain identifier handling, which informed several refutations in the report.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.